

DE-GNN: Dual embedding with graph neural network for fine-grained encrypted traffic classification

Xinbo Han^{a,b}, Guizhong Xu^c, Meng Zhang^{a,b}, Zheng Yang^{a,b}, Ziyang Yu^{a,b}, Weiqing Huang^{a,b}, Chen Meng^{a,*}

^a Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China

^b School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China

^c Information Science Academy of China Electronics Technology Group Corporation, Beijing, China

ARTICLE INFO

Keywords:

Encrypted traffic classification
Deep learning
Graph neural networks
Dual-embedding
Traffic interaction graph

ABSTRACT

Nowadays, most network traffic is encrypted, which protects user privacy but complicates the task of analyzing and classifying encrypted traffic. Identifying the specific categories of encrypted traffic, such as application type or even the specific application, is of great significance for advanced network services and network security management. Many existing methods for encrypted traffic classification rely on machine learning and deep learning techniques, but they exhibit certain shortcomings. A considerable number of these methods rely on statistical features, which may lose their relevance as networks evolve and lead to the loss of important information. Additionally, Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) face limitations in extracting features from encrypted traffic, specifically, their inability to learn traffic interaction information within a network flow.

To address these challenges, we propose a model called Dual Embedding with Graph Neural Networks (DE-GNN) for fine-grained encrypted traffic classification. Based on the byte-packet-flow structure of network traffic, we present a dual embedding layer that encodes the packet header and payload separately using raw bytes, which allows subsequent processes to run separately and in parallel. Then, we develop the PacketCNN to extract packet-level features from both the header and payload. Afterwards, we construct a network flow as a Traffic Interaction Graphs (TIG) and utilize Graph Neural Networks (GNNs) to extract flow-level features. Finally, an adaptive deep feature fusion process is applied to combine flow-level features from the header and payload, creating a robust representation for classification. Extensive experiments are conducted on two well-known datasets, ISCX-VPN2016 and ISCX-Tor2016, to verify our approach. The experimental results demonstrate that DE-GNN effectively identifies the type of encrypted traffic, outperforming baselines significantly.

1. Introduction

Identifying traffic types is a fundamental and critical ability for network research. Traffic classification, the categorization of network traffic into appropriate classes, is important to many applications, such as quality of service (QoS) control, pricing, resource usage planning, malware detection, and intrusion detection [1]. In recent times, to protect user privacy and anonymity, various encryption techniques are used to encrypt the transmission of network traffic [2]. While this ensures the privacy and anonymity for users, it complicates the task of analyzing and classifying network traffic. Furthermore, privacy-enhanced tools, such as VPN and Tor [3], are employed to circumvent Internet censorship measures, which further increase the difficulty of

classifying encrypted traffic. Consequently, the task of extracting valuable information from encrypted traffic presents a formidable challenge in the pursuit of accurate classification.

Early traffic classification methods included port-based techniques and Deep Packet Inspection (DPI) [4–6]. However, with the proliferation of applications utilizing non-standard interfaces or dynamically assigned ports, the effectiveness of port-based methods has significantly waned. The Deep Packet Inspection (DPI) method involves a thorough examination and analysis of network packets to detect and identify specific protocols, applications, content, or behaviors within network traffic. While DPI has outperformed port-based detection methods, it does

* Corresponding author.

E-mail addresses: hanxinbo@iie.ac.cn (X. Han), xuguizhong@cetc.com.cn (G. Xu), zhangmeng@iie.ac.cn (M. Zhang), yangzheng@iie.ac.cn (Z. Yang), yuziyang@iie.ac.cn (Z. Yu), huangweiqing@iie.ac.cn (W. Huang), mengchen@iie.ac.cn (C. Meng).

<https://doi.org/10.1016/j.comnet.2024.110372>

Received 21 November 2023; Received in revised form 26 February 2024; Accepted 25 March 2024

Available online 27 March 2024

1389-1286/© 2024 Elsevier B.V. All rights reserved.

incur substantial computational overhead and faces certain limitations when dealing with encrypted traffic [7]. To address these challenges, some have proposed a two-step approach, combining traffic decryption with DPI [8–11]. Nevertheless, this approach undeniably amplifies the computational load and raises concerns about privacy implications, which contradicts the original purpose of traffic encryption.

The advancement of artificial intelligence technology has ushered in a new era of traffic classification methods, primarily centered on the utilization of machine learning and deep learning algorithms. Machine learning approaches employing statistical features have shown promising outcomes for both encrypted and non-encrypted traffic [12–14]. However, these algorithms heavily rely on manually crafted features by domain experts. As network dynamics evolve, previously designed features may lose their effectiveness, thus imposing significant constraints on machine learning-based traffic classification methods. Additionally, statistical features often necessitate observing the entire network flow, making them incapable of classifying traffic during the initial stages of network traffic generation [1]. As a branch of machine learning, deep learning has demonstrated substantial potential in the realm of encrypted traffic classification [15–17]. Deep learning techniques no longer depend on manual feature engineering, as they can automatically extract relevant features from raw data. Embracing an end-to-end learning paradigm, deep learning directly learns from the original input data to produce the desired output, bypassing traditional practices of decomposing the problem into smaller subproblems such as feature selection and classification. This characteristic empowers deep learning algorithms to comprehensively tackle the complexities associated with traffic classification tasks [1]. In light of the instability of statistical features, we use raw bytes as the input of the model in this paper.

The current encrypted traffic classification methods based on deep learning have demonstrated remarkable capabilities. Many of these methods harness the power of Convolutional Neural Networks (CNN) and Recurrent Neural Networks (RNN) to capture spatial and temporal features within traffic data. Nevertheless, two primary flaws persist in current deep learning-based approaches: **(1) Limited information capture.** Due to the encryption of information, CNN and RNN exhibit constraints in effectively extracting data from encrypted traffic. Additionally, CNN and RNN focus on temporal and spatial characteristics, often overlooking traffic interaction mode. **(2) Mix usage of the header and payload.** Approaches employing packet bytes consider the header and payload as a unified entity for learning, without differentiating their distinct representations and functions.

In this paper, we propose a model called Dual Embedding with Graph Neural Network (DE-GNN) for encrypted traffic classification. DE-GNN primarily comprises four main components, as illustrated in Fig. 1: Dual Embedding, Packet-level Traffic Learning, Flow-level Graph Construction and Learning, and Adaptive Deep Feature Fusion. Based on the byte-packet-flow structure of network traffic, the dual embedding treats the header and payload bytes of a packet separately and encodes them using one-hot encoding. Subsequently, two separate PacketCNNs are employed to extract packet-level features for both the header and payload. Following this, flow-level features are obtained through the construction of the Traffic Interaction Graph (TIG) and learning from GNN. The flow-level features of the header and payload are then integrated into a comprehensive vector for the entire flow using adaptive deep feature fusion, which is used for traffic classification. In the experimental section, we employ two publicly available datasets, namely, ISCX-VPN and ISCX-Tor, to conduct a series of experiments aimed at demonstrating the validity and effectiveness of our model.

In summary, the contributions of this paper are as follows:

- We propose DE-GNN, a model that separately processes packet headers and payloads and employs a hierarchical structure for a comprehensive study of fine-grained encrypted traffic classification.
- We utilize a CNN structure to learn packet-level representations from raw bytes, reducing the impact of noise and redundant data compared to using the original byte vectors directly.
- We construct network flows in the form of a Traffic Interaction Graph, incorporating information on behavioral patterns and packet-level features, and leverage Graph Neural Networks to acquire flow-level features.
- To assess the effectiveness of our model, we perform a series of experiments. The experimental results affirm the efficacy of our approach.

The rest of the paper is organized as follows: Section 2 presents the task description and the related works on encrypted traffic classification; Section 3 describes our model in detail; Section 4 introduces relevant experiments and analyzes the results; We discuss limitations and future works in Section 5 and conclude the paper in Section 6.

2. Task description and related works

At present, a significant volume of research in the field of encrypted traffic classification centers on the utilization of machine learning and deep learning models. In the subsequent section, we describe the encrypted traffic classification task and briefly review existing studies.

2.1. Task description

The encrypted traffic classification task aims to differentiate the traffic generated from various sources (e.g., applications, web pages, or services) by using the information of traffic packets captured by professional software or programs. In this paper, our focus is on classifying in-app traffic behavior, distinguishing fine-grained encrypted traffic actions such as video, chat, and file transfer.

We define a flow as an sample, a common practice in many traffic classification studies [18–21]. A flow comprises request and response packets exchanged between two communicating entities in a network, sharing the same quintuple (src_ip, src_port, dst_ip, dst_port, protocol) over a specific period. Assume that there are M training samples and N categories in total, let a flow sample be a sequence $l = [p_1, p_2, \dots, p_n]$, where n is the packet sequence length and p_i is the i th packet of the flow. The i th packet is represented as $p_i = [b_1, b_2, \dots, b_m]$, where m is the byte sequence length and b_j is the j th byte value of i th packet. In other words, network traffic exhibits a hierarchical structure: byte-packet-flow. According to the definition above, the encrypted classification task can be described formally as predicting the category C_s of an unseen test sample f_s with a designed and well-trained end-to-end model $F(f_s)$ on M training samples, where $C_s = 0, 1, \dots, N - 1$.

2.2. Related works

Machine learning based encrypted traffic classification. Numerous methods within the realm of encrypted traffic classification leverage statistical features to characterize packet or flow properties, employing classical machine learning models for the task of traffic classification. Taylor et al. [22] introduced AppScanner, a classification model that utilizes the random forest algorithm to distinguish mobile apps based on statistical features extracted from flow data. Liu et al. [23] proposed a multi-attribute Markov probability model, which integrates message type sequence and packet length, to classify traffic applications. Ede et al. [24] provided a method called FlowPrint for identifying mobile applications, which combines destination-based clustering, browser isolation, and pattern recognition in a semi-supervised manner to perform application fingerprinting. Taylor et al. [25] explored how a passive eavesdropper can discern smartphone apps by analyzing the network traffic they transmit. The authors employ machine learning techniques to identify smartphone apps based on side-channel data, including packet size and direction, despite the payload of packets

being concealed by SSL/TLS encryption. Shen et al. [20] introduced a webpage fingerprinting method called FineWP, which involves the accumulation of packet lengths and extraction of block, sequence, and statistical features. These features are subsequently input into machine learning algorithms such as random forest, decision tree, and k-nearest neighbors to create webpage fingerprints. Saber et al. [26] proposed a combination of Principal Component Analysis (PCA) and support vector machines for traffic classification, focusing exclusively on time-based flow features. Anderson et al. [27] proposed the utilization of the logistic regression algorithm to jointly analyze flow-level features, header information, and additional features for the purpose of classifying encrypted traffic into either malicious or normal traffic categories. These methods typically necessitate manual feature selection and face challenges due to feature instability.

Deep learning based encrypted traffic classification. Deep learning has gained extensive prominence in traffic classification tasks owing to its inherent capability for automated feature extraction. Liu et al. [28] introduced FS-Net, a framework designed for encrypted traffic classification via deep neural networks. FS-Net takes the packet length sequence as its input, employs bi-directional GRU [24] for feature encoding, and incorporates a reconstruction mechanism within an AutoEncoder to guarantee the validity of the learned features. Shapira et al. [29] proposed a novel approach for encrypted Internet traffic classification and application identification by transforming basic flow data into an intuitive picture, a FlowPic, and then using Convolutional Neural Networks to identify the flow category and the application in use. Liu et al. [30] introduced a model named BGRUA to identify the web services running on HTTPS connections. BGRUA employs a bi-directional gated recurrent unit (GRU) to capture both forward and backward features from the byte sequences within a session. Additionally, an attention mechanism is utilized to assign weights to features based on their respective contributions to the classification task. Lin et al. [31] proposed a new traffic representation model called Encrypted Traffic Bidirectional Encoder Representations from Transformer (ET-BERT), which pre-trains deep contextualized datagram-level representation from large-scale unlabeled data. They proposed two pre-training tasks capture the contextual relationship between traffic bytes by predicting the masked token as well as the correct transmission order by predicting the Same-origin BURST. Mohammad et al. [32] proposed a deep learning-based approach called “Deep Packet” for encrypted traffic classification, which utilizes two deep neural network structures, stacked autoencoder (SAE) and convolution neural network (CNN) for application identification tasks. Aceto et al. [33] proposed a multi-modal deep learning method called MIMETIC. They exploited CNN and GRU respectively to learn two different modalities, the first 576 bytes of payload and informative protocol fields, for mobile encrypted traffic classification. Wang et al. [34] also introduced a similar model called App-Net, which use LSTM and CNN to learn sequence of packet lengths and payload bytes of the initial packet respectively. Graph neural networks (GNN) have strong potential in processing unstructured data and demonstrate promising performance in many fields. Shen et al. [35] proposed GraphDApp, which can identify encrypted traffic of Decentralized Applications (DApps) using GNNs. They constructed the Traffic Interaction Graph (TIG) of encrypted flows based on the packet length and packet direction and turned the DApp identification into a graph classification problem. Huoh et al. [36] proposed a graph construction method, which directly create edges based on the chronological relationship of packets among a flow and leverage the geometric learning model [37] to learn the node, edge, and global features.

We summarize related deep learning based works in Table 1 and list our main novelty compared with these methods:

(1) Hierarchical structure: Previous research commonly utilizes statistical features or raw bytes directly to represent network traffic and employs various learning models. In this paper, we conduct hierarchical learning on raw bytes, exploring rich information following the sequence of byte-packet-flow.

(2) Separate learning of packet headers and payloads: Most methods overlook the distinction between packet headers and payloads. Our method independently learns both headers and payloads.

(3) Traffic behavioral patterns: In addition to utilizing CNN for learning information from raw bytes, we also focus on the interaction behavior between both parties. We construct a Traffic Interaction Graph and apply GNN for graph learning.

3. Methodology

In this section, we will provide a comprehensive description of our model. The overall model is shown in Fig. 1, which mainly includes dual embedding, packet-level traffic learning, flow-level graph construction and learning, and adaptive deep feature fusion.

3.1. Dual embedding

In our classification task, we focus on network flows. We opt for raw bytes to represent packets rather than employing statistical features because raw bytes contain richer and more comprehensive information. In general, distinct byte values convey distinct meanings. Two bytes with different values correspond to two distinct embedding vectors. However, in the context of encrypted traffic, the interpretation of a byte is not solely contingent upon its value but also its position within the packet, specifically, whether it belongs to the header or payload.

Encrypted traffic exhibits notable disparities between the header and payload. These differences manifest in several ways. **Different representations:** The header of a packet is typically plaintext information that remains unencrypted, whereas the payload contains ciphertext information resulting from encryption. **Different functions:** Packet headers primarily include information such as the five-tuple and protocol type, serving a control and descriptive role during transmission. In contrast, payloads predominantly carry user data, constituting the specific content of communication between both parties. Given these distinctions, it becomes apparent that identical byte values in the header and payload carry different meanings. If we were to mix the header and payload and learn together, it would introduce difficulties for a model to converge to optimal values. This is due to the ambiguity in the meaning associated with these bytes when they have identical representations. For the aforementioned reasons, we opt to learn the header and payload separately, with the aim of reducing confusion throughout the data learning process.

One-hot encoding. While each byte can be quantified as a numerical value, it is essential to note that these values within network traffic lack specific numeric meaning or continuity. Consequently, we employ one-hot encoding to represent each byte. One-hot encoding generates a unique vector representation for each byte value, where only a single bit in the vector is set to 1, while all other bits are set to 0. This encoding technique is widely utilized in machine learning to represent discrete values. Given that there are 256 possible values for a byte, we represent each byte as a vector with a dimensionality of 256.

Dual embedding layer. For the packet header, we select the first K_h bytes from each packet header and employ one-hot encoding to represent it, denoted as $E_h \in \mathbb{R}^{K_h \times 256}$. Concerning the packet payload, we choose K_p bytes and obtain its representation, denoted as $E_p \in \mathbb{R}^{K_p \times 256}$. To address situations where the length is insufficient, we apply zero-padding, and in cases where the length exceeds the expected value, we perform truncation. Consequently, we have now obtained the dual embedding of the header and payload, represented as E_h and E_p , where each row signifies an individual byte.

Since we process the header and payload separately, all subsequent modules are likewise treated independently. While the network structures are the same, they do not share parameters and can be concurrently processed in parallel.

Table 1

Comparison with recent related works.

Reference	Algorithm	Network Input	Raw bytes	Separate learning of header and payload
Liu et al. [28]	AE+GRU	Packet length	–	–
Shapira et al. [29]	CNN	IP packet length, time of arrival	–	–
Liu et al. [30]	GRU	Payload bytes	✓	–
Lin et al. [31]	Bert	Packet bytes	✓	–
Mohammad et al. [32]	SAE+CNN	Packet bytes	✓	–
Aceto et al. [33]	CNN+GRU	Payload bytes, protocol fields	✓	–
Wang et al. [34]	CNN+LSTM	Payload of the initial packet, packet length	✓	–
Shen et al. [35]	GNN	Packet length, packet direction	–	–
Huoh et al. [36]	GNN	Raw bytes, inter-arrival time, meta features of a flow	✓	–
This paper	GNN+CNN	Header raw bytes, payload raw bytes	✓	✓

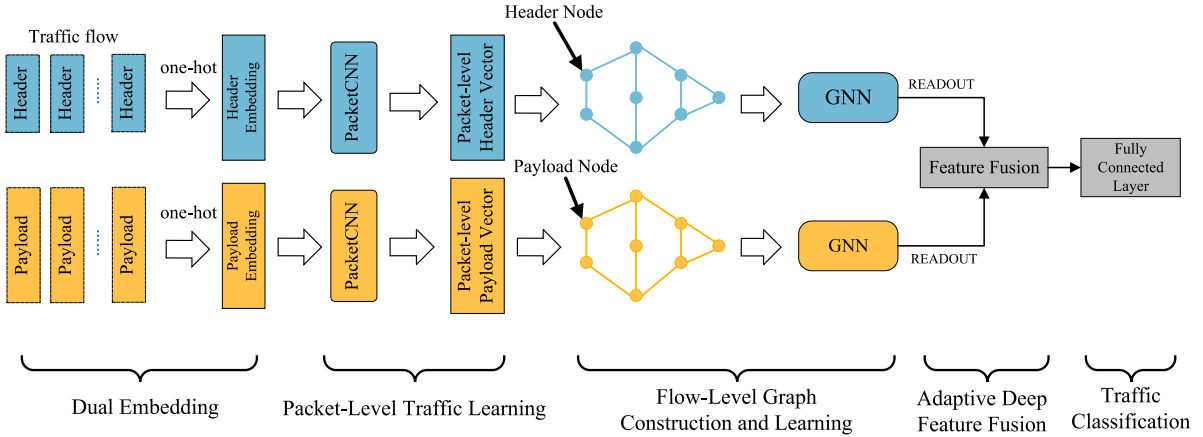


Fig. 1. DE-GNN model architecture. (a) Dual embedding: Encode packet header and payload separately using raw bytes; (b) Packet-level traffic learning: Learn package-level features using PacketCNN; (c) Flow-level graph construction and learning: Build traffic interaction graphs and learn flow-level features using GNN; (d) Adaptive deep feature fusion: Fuse the packet header and payload features; (e) Traffic classification: Obtain the label of encrypted traffic.

3.2. Packet-level traffic learning

Given the hierarchical structure inherent in network flow data (i.e., byte-packet-flow), our objective is to perform a deep analysis of the rich content within network traffic flow. To achieve this, we commence by learning network traffic at the packet level.

Network traffic packets are composed of bytes, and we have successfully represented the packet using bytes. To extract abstract features within a packet, we employ a byte extraction model, enabling us to learn the packet-level representation of network traffic. At this stage, every byte within the packet is represented in a word-like manner. Drawing inspiration from the TextCNN model [38], we design a similar structure, PacketCNN, to process each packet. The model's architecture is visually depicted in Fig. 2.

We employ three different convolutional kernels to extract the underlying content within each packet respectively. Each convolution operation consists of a convolutional layer, the ReLU activation functional, and a max pooling layer. To illustrate, let us consider the payload E_p as an example:

$$h_p^{(1)} = \text{MaxPool}(\text{ReLU}(\text{Conv}(E_p))) \quad (1)$$

Using three convolution kernels, we obtain three vectors, denoted as $h_p^{(1)}, h_p^{(2)}, h_p^{(3)} \in \mathbb{R}^d$. Subsequently, we utilize the concatenation operation and a fully connected layer for amalgamating the outcomes from three convolutional kernels, with the ReLU activation function applied.

$$h_p^{final} = \text{CONCAT}(h_p^{(1)}, h_p^{(2)}, h_p^{(3)}) \quad (2)$$

$$F_p^{packet} = \text{ReLU}(\text{FC}(h_p^{final})) \quad (3)$$

where F_p^{packet} is the packet-level representation of payload. The pro-

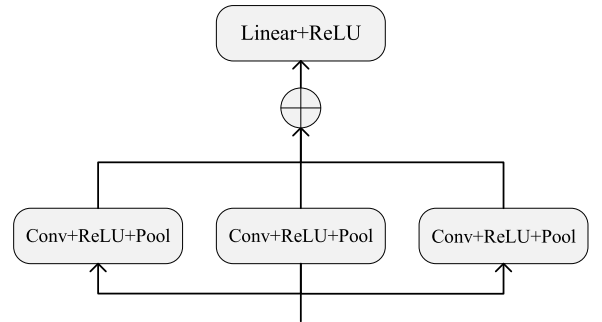


Fig. 2. The structure of PacketCNN. PacketCNN uses three different convolution operations to extract information from packets.

cessing of the header follows a similar approach.

As of now, we have obtained packet-level representations for both the packet header and payload, denoted as F_h^{packet} and F_p^{packet} , each with dimensions of $(M, 1)$. M is the dimension of the obtained packet-level representation from PacketCNN learning. These acquired packet-level representations will subsequently serve as the feature representations for nodes within the Graph.

In contrast to directly utilizing the original byte values to represent the header and payload, this method enables us to delve deeper into the effective characteristics of the packet. As a result, it mitigates the influence of noise and redundant information, culminating in a more robust and informative representation. This is further supported by the results of the ablation experiment.

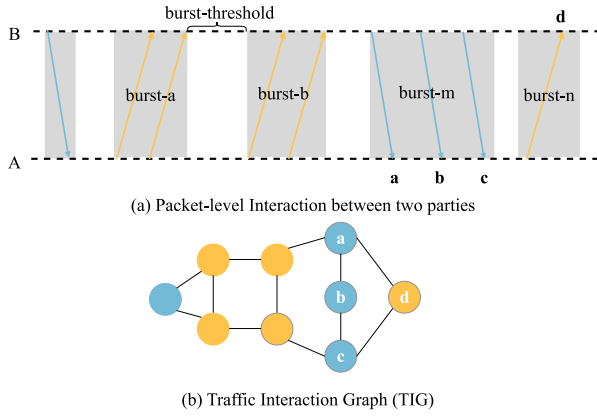


Fig. 3. Construction of traffic interaction graph. Fig. 3(a) illustrates the communication process between parties A and B. Each line represents a packet, and the arrow indicates the direction of transmission. Fig. 3(b) depicts the traffic interaction graph constructed based on (a), where each node represents a packet. Different boxes represent different bursts.

3.3. TIG construction and flow-level traffic learning

3.3.1. Traffic interaction graph (TIG) construction

Due to the reduction in effective features within encrypted traffic, traditional traffic classification methods reliant on plaintext content have experienced a decline in performance. However, it is imperative to recognize that while encryption protocols secure the content of traffic, they do not alter the interaction mode of the communicating parties. Each type of traffic retains its distinct interaction pattern. Each traffic flow, consisting of a series of packets resulting from client-server interactions, can be represented by an information-rich graph structure named Traffic Interaction Graph (TIG) [35]. Through the construction of the TIG, we can effectively depict the traffic interaction mode between the two communicating parties.

As depicted in Fig. 3(a), it provides a schematic representation of communication between the two sides of the network. In this depiction, each line within the graph symbolizes a traffic packet, with the direction of the arrow indicating the transmission direction of the packet. This depiction is further transformed into a graph structure, as illustrated in Fig. 3(b). First, we introduce the concept of burst. A burst is defined as a sequence of consecutive packets transmitted along the same direction during a short time interval [39], and the current packet occurs within a threshold of time, the burst threshold, of the previous packet [25]. As shown in Fig. 3(a), each box represents a burst. Although burst-a and burst-b are the same continuous packet sequence, their temporal distance exceeds the burst threshold, so they are divided into two bursts.

TIG is represented by vertices and edges:

Vertices: In a network flow, each packet is treated as an individual entity. Consequently, each vertex in the graph signifies a packet. Specifically, we consistently utilize the previously acquired packet-level representation to signify each packet, i.e., F_h^{packet} or F_p^{packet} .

Edges: TIG involves segmenting a network flow into multiple bursts and then employing the relationships between these bursts to represent the connections of vertices. There are two types of edges: (1) Intra-Burst Edges. This type of edge is used to link multiple vertices within the same burst. It establishes connections between these vertices in chronological order according to their respective timestamps. For example, in burst m , there exist two intra-burst edges, connecting vertices a and b , as well as vertices b and c . (2) Inter-Burst Edges. These edges are employed to establish connections between two consecutive bursts. More specifically, the first and last vertices in each burst are linked to the corresponding first and last vertices in its preceding burst. For instance, between burst m and burst n , there is a connection between

vertex a and vertex d , as well as between vertex c and vertex d . It is worth noting that as burst n comprises only one vertex, both its first and last vertices are represented by vertex d . It is important to emphasize that in order to facilitate bidirectional information propagation, we employ undirected edges within the Graph.

Below, we provide the symbol definitions and algorithm descriptions for TIG.

Definition 1 (Traffic Interaction Graph). A TIG is defined by a four-tuple $(V, E, F, label)$:

- V is a set of vertices, where each vertex $v \in V$ represents an individual packet within the network flow.
- E is a set of edges used to depict the connectivity relationships among vertices, including two kinds of edges: intra-burst edges and inter-burst edges. Each edge $e \in E$ can be categorized into one of these two types.
- F represents the set of vertex features, which constitutes the packet-level representations that we previously obtained.
- $label$ stands for the label of the TIG, which corresponds to the category assigned to the network flow.

Algorithm 1 Construction of Traffic Interaction Graph (TIG)

Input: The sequence of packet direction $D = \{d_1, d_2, \dots, d_N\}$, packet arrival time $S = \{s_1, s_2, \dots, s_N\}$, packet-level feature representation $F^{packet} = \{f_1, f_2, \dots, f_N\}$ and $burst_threshold$

Output: The traffic interaction graph $G = (V, E, F)$

```

1: Initialize  $V$ ,  $E$ , and  $F$  as empty sets
   {Add vertices and corresponding node features}
2: for all  $d_i \in D$ ,  $f_i \in F^{packet}$  do
3:   Add a vertex  $d_i$  in  $V$  and configure its node feature  $f_i$  in  $F$ 
4: end for
   {Separate  $V$  into bursts set  $B = \{b_1, b_2, \dots, b_K\}$  according to packet direction and timestamp}
5: Create an empty burst, denoted as  $b$ , and add vertex  $d_1$  to it. Set both the first and last vertices of  $b$  to  $d_1$ .
6: for  $i = 2$  to  $N$  do
7:   if  $d_i == d_{i-1}$  and  $(s_i - s_{i-1}) < burst\_threshold$  then
8:     Add vertex  $d_i$  to burst  $b$  and update the last vertex to  $d_i$ 
9:   else
10:    Burst  $b$  is complete and add it to  $B$ 
11:    Create a new empty burst and add vertex  $d_i$  to it. Set both the first and last vertices to  $d_i$ 
12:   end if
13: end for
   {Add intra-burst edges and inter-burst edges}
14: for all  $b_i \in B$  do
15:   Add intra-burst edges inside  $b_i$  and inter-burst edges between  $b_i$  and  $b_{i+1}$  in  $E$ 
16: end for
17: return  $G$ 

```

The construction of the Traffic Interaction Graph (TIG) follows the outlined procedure as presented in Algorithm 1. The packet direction sequence $D = \{d_1, d_2, \dots, d_N\}$, where $d_i \in \{1, -1\}$ (1 indicating the sending direction and -1 indicating the receiving direction), packet arrival time $S = \{s_1, s_2, \dots, s_N\}$, where s_i indicates the packet arrival time in a flow, the previously obtained packet-level features F^{packet} , along with $burst_threshold$ serves as the input. The algorithm proceeds to build the vertex set and configure vertex features using F^{packet} (lines 2–4). Subsequently, it segments the network flow into multiple bursts (lines 5–13) and introduces two types of edges connecting the vertices: intra-burst edges and inter-burst edges (lines 14–16).

TIG encapsulates two critical types of information:

Behavioral pattern information: Each category of encrypted traffic exhibits unique behavioral patterns. TIG effectively represents the interaction behavior between both parties engaged in network communication. This includes behavioral details such as packet burst characteristics and packet sequence information. These patterns are amenable to extraction and analysis using graph neural networks to derive effective features. **Packet-level traffic information:** This involves using the previously acquired packet-level features as vertex features. These features encapsulate valuable information gleaned from the packets. In contrast to using packet length and packet direction as vertex attributes, this approach offers a more comprehensive and information-rich representation.

3.3.2. Learning flow-level features using graph neural network

We leverage TIG to characterize the behavioral patterns of network traffic and employ Graph Neural Networks (GNNs) for feature extraction. In contrast to Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs), which predominantly focus on spatial and temporal features, GNNs offer the advantage of more effectively capturing structural information from graphs. Additionally, CNNs and RNNs often necessitate alignment in the number of packets, which can lead to the introduction of redundant zero elements when handling short flows, potentially obscuring valuable information. In contrast, graph-based representations do not require such packet number completion, thereby reducing the introduction of noise.

More specifically, we utilize the Graph Attention Network (GAT) [40] for information extraction from the TIG. Given that each vertex in the TIG represents a packet and holds varying significance within the overall graph structure, GAT leverages the attention mechanism to assign distinct weights to different nodes in the neighborhood. This enables neighboring nodes to carry varying degrees of importance during the message propagation process, thereby accentuating information from critical nodes while dampening the influence of less relevant nodes.

Given a TIG, its node features are $\mathbf{h} = \{h_1, h_2, \dots, h_N\}$, $h_i \in \mathbb{R}^M$, with N as the number of nodes and M as the dimension of features in each node. Firstly, an attentional mechanism, a , is used to calculate the attention coefficients:

$$e_{ij} = a(\mathbf{W}h_i, \mathbf{W}h_j) \quad (4)$$

that indicate the importance of node j 's features to node i , where $\mathbf{W}$ is a learnable weight matrix. We only compute e_{ij} for nodes $j \in \mathcal{N}_i$, where $\mathcal{N}_i$ represents all neighbors of node i in the graph.

To ensure that coefficients are easily comparable across different nodes, GAT normalizes them across all possible choices of j using the softmax function:

$$\alpha_{ij} = \text{softmax}_j(e_{ij}) = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}_i} \exp(e_{ik})} \quad (5)$$

Subsequently, the obtained attention coefficients are employed to update the node features.

$$h_i^{(1)} = \sigma\left(\sum_{j \in \mathcal{N}_i} \alpha_{ij} \mathbf{W}h_j\right) \quad (6)$$

where σ is the activation function, and $\mathbf{W}$ is a learnable weight matrix. Furthermore, the utilization of multi-head attention mechanisms can contribute to the stabilization of the learning process.

The GAT layer produces a new set of node features (of potentially different cardinality M'), $\mathbf{h}^{(1)} = \{h_1^{(1)}, h_2^{(1)}, \dots, h_N^{(1)}\}$, $h_i^{(1)} \in \mathbb{R}^{M'}$.

Finally, we apply mean pooling on all nodes to get a graph feature vector g :

$$g^{(1)} = \frac{h_1^{(1)} \oplus h_2^{(1)} \oplus \dots \oplus h_N^{(1)}}{N} \quad (7)$$

To thoroughly extract information from the TIG, we employ a stack of multiple GAT layers (in this paper, 4 layers are utilized), as shown in

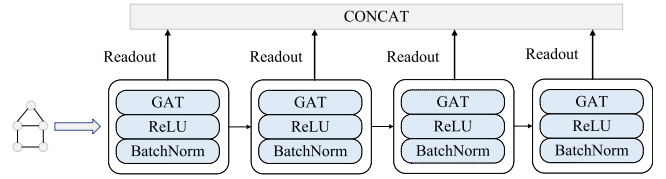


Fig. 4. Structure of GNN. We stack 4 GAT layers, with each GAT layer generating a graph feature vector. These 4 graph feature vectors are concatenated together to form the flow-level features.

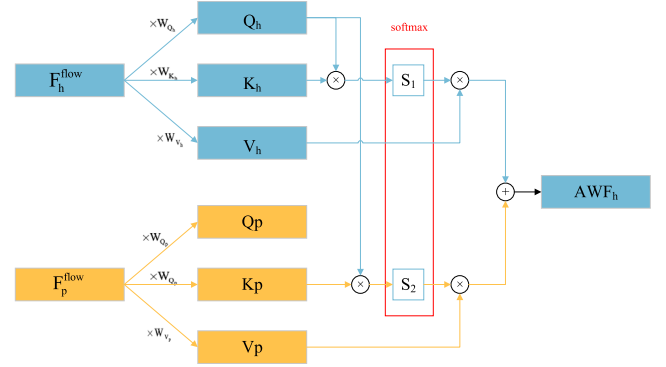


Fig. 5. Adaptive-weighted sum for packet header.

Fig. 4. Following each layer, we apply the ReLU activation function and BatchNorm. We acquire four graph feature vectors: $g^{(1)}, g^{(2)}, g^{(3)}, g^{(4)}$, and subsequently concatenate them as the flow-level features:

$$F^{flow} = \text{CONCAT}(g^{(1)}, g^{(2)}, g^{(3)}, g^{(4)}) \quad (8)$$

At this point, we have obtained flow-level representations for both the packet header and payload, denoted as F_h^{flow} and F_p^{flow} , with dimensions of $(S, 1)$ both. S is the dimension of flow-level representation from GNN learning.

3.4. Adaptive deep feature fusion

Via the construction of TIG and the learning of GNN, we have acquired the flow-level feature representations for both the packet header and payload. These features are subsequently employed for traffic classification. While header and payload contribute differently to the classification, it is necessary to use a feature fusion strategy to jointly learn the features from them to accurately and robustly classify traffic. However, most of the existing feature fusion strategies are based on fixed combination weights, including direct concatenation [41], and weighted concatenation [42]. To combine the features from the packet header and payload, we employ adaptive deep feature fusion (ADFF) [43].

Let us take the header as an example. As shown in Fig. 5, the ADFF mechanism first calculates a query vector $Q_i = F_i^{flow} \times W_{Q_i}$, a key vector $K_i = F_i^{flow} \times W_{K_i}$, and a value vector $V_i = F_i^{flow} \times W_{V_i}$ for each feature ($i = \{h, p\}$, denoted as header and payload). Then, ADFF calculates the dot products of Q_h (representing the query vector of header) with all keys K_i to generate two corresponding scores, labeled as S_1 and S_2 . To obtain a normalized weight score, ADFF applies the softmax function to the scores. Finally, ADFF computes the dot products of the value vectors with their respective weights and then sums them to obtain the weighted header feature vector, denoted as AWF_h :

$$AWF_h = \sum_{i=\{h,p\}} \text{Softmax}(Q_h \cdot K_i) V_i \quad (9)$$

Similarly, we get the weighted payload feature vector AWF_p . Finally, as demonstrated in Fig. 6, the fused feature vector F_{final} is obtained by

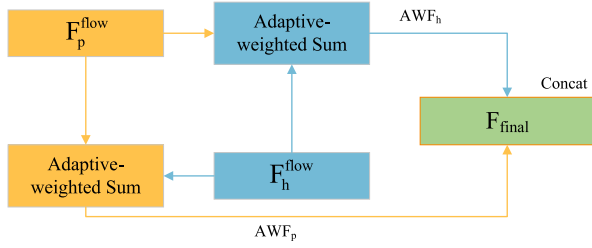


Fig. 6. Adaptive deep feature fusion. We utilize adaptive deep feature fusion to merge the header and payload.

concatenation of the two weighted feature vectors:

$$F_{final} = \text{CONCAT}(AWF_h, AWF_p) \quad (10)$$

3.5. Classification

We obtain the final flow representation, denoted as F_{final} , and employ it for multiple classification tasks. We conduct a fully connected layer followed by a softmax operation on it to obtain the predicted results. Subsequently, we utilize cross-entropy as the loss function to calculate the loss, which is subsequently employed for the backward pass during training.

$$\hat{y} = \text{Softmax}(W \cdot F_{final} + b) \quad (11)$$

$$\text{loss} = - \sum_{i=1}^K y_i \log \hat{y}_i \quad (12)$$

where $\{W, b\}$ are the parameters of the corresponding fully connected layer, $\hat{y}$ is the predicted probability values, and K is the number of traffic categories.

4. Experiments and analysis

In this section, we introduce our experimental setup, including dataset, traffic pre-processing, and evaluation metrics. Then, we conduct a series of experiments, including model sensitivity analysis, comparison experiments, ablation experiments, GNN architecture variants study, TIG structure study, and model complexity analysis which evaluate the performance of DE-GNN fully and specifically.

4.1. Experimental setup

4.1.1. Dataset

To comprehensively evaluate the effectiveness of our model, we utilize multiple public datasets, namely ISCX-VPN2016 [44] and ISCX-Tor2016 [45].

ISCX-VPN2016 contains encrypted traffic encompassing various types, including VoIP, P2P, and more. It is divided into two parts: ISCX-VPN and ISCX-nonVPN datasets. The ISCX-VPN dataset comprises traffic collected over virtual private networks (VPNs), which are commonly used to access blocked websites or services. Recognizing VPN traffic can be challenging due to obfuscation technologies. In contrast, the ISCX-nonVPN dataset consists of regular encrypted traffic not collected over VPNs. It is worth noting that due to label confusion of traffic files, we categorize both VPN and nonVPN datasets into 6 distinct category labels as referenced in [46]. The labels are shown in Table 2.

Similarly, ISCX-Tor2016 is another public dataset, comprising ISCX-Tor and ISCX-nonTor datasets. The ISCX-Tor dataset contains traffic collected over the onion router (Tor), which can be difficult to trace. The ISCX-nonTor dataset consists of regular encrypted traffic not collected over Tor. Please note that, according to official instructions, the

Table 2

Labeled ISCX-VPN2016 and ISCX-Tor2016 datasets. This table illustrates the traffic types we divide the datasets into and the content contained in different categories.

Dataset	Traffic type	Content
ISCX-VPN	VPN-Email	Email, Gmail(SMTP, POP3,IMAP)
	VPN-Chat	ICQ, AIM, Skype, Facebook, Hangouts
	VPN-Streaming	Vimeo, Youtube, Netflix, Spotify
	VPN-File transfer	Skype, FTPS, SFTP
	VPN-VoIP	Facebook, Skype, Hangouts, Voipbuster
	VPN-P2P	uTorrent, Bittorrent
ISCX-nonVPN	Email	Email, Gmail(SMTP, POP3,IMAP)
	Chat	ICQ, AIM, Skype, Facebook, Hangouts
	Streaming	Vimeo, Youtube, Netflix, Spotify
	File transfer	Skype, FTPS, SFTP
	VoIP	Facebook, Skype, Hangouts, Voipbuster
	P2P	uTorrent, Bittorrent
ISCX-Tor	Tor-Browsing	Firefox, Chrome
	Tor-Email	Thunderbird (SMTP/S, POP3/SSL, IMAP/SSL)
	Tor-Chat	Facebook, Hangouts, Skype, IAM, ICQ
	Tor-Audio	Spotify
	Tor-Video	YouTube, Vimeo
	Tor-FTP	Skype, SFTP and FTPS
	Tor-VoIP	Facebook, Hangouts, Skype
	Tor-P2P	Bittorrent

nonTor dataset utilizes traffic from the VPN project, and detailed label allocation instructions are lacking. To ensure experimental rigor, we only use the Tor dataset, which is subdivided into 8 distinct category labels. The labels are shown in Table 2.

4.1.2. Pre-processing

The datasets provide original traffic files (Pcap format). We need to preprocess the Pcap files in the datasets. Our operations on the dataset can be mainly summarized as: **(1) PCAP segmentation.** The traffic classification task is finally carried out on the unit of flow, so we split the Pcap files into bi-flows using a flow segmentation tool (such as Splitcap) according to the quintuple information (source IP, source port, destination IP, destination port, protocol). Following segmentation, data cleaning is carried out to remove duplicates and invalid samples [41]. **(2) Header and payload extraction.** We extract the header and payload of each packet separately and convert each byte into one-hot encoding. The purpose is to process flow into a pattern acceptable to the neural network. **(3) Labeling.** We assign labels to each sample by referring to the official instructions and previous research.

During this process, we exclude the Ethernet header, as it only contains irrelevant information for classification purposes. Additionally, the source and destination IP addresses and ports are all removed for the purpose of eliminating interference with sensitive information deriving from them. Furthermore, we perform truncation on excessively long data and apply zero-padding to data with an insufficient quantity.

4.1.3. Implementation details

There are some critical hyperparameters that need to be set in the model. Table 3 shows our hyperparameter values for the best classification performance. We use a grid search and conduct the 5-fold cross-validation on the training set to select the best hyperparameters. During the model training, we use the EarlyStopping which ensures that the learning is terminated without reaching the extreme fit of the model.

To conduct the 5-fold cross-validation, the training datasets are randomly split into 5 equal-size subsets. In each fold of the cross-validation, 4 subsets (80%) of the data are used for training a brand new model, and the rest subset (20% of the data), different in each fold, is used to evaluate the resultant model. For a particular set of model parameters, when all 5 training-validation folds are finished, we compute the final validation loss by averaging the 5 validation losses

Table 3

Hyperparameter values of DE-GNN.

HyperParameter	Value
The convolution kernel sizes of PacketCNN	(2,3,4)
The number of convolution kernels of PacketCNN	32
The dimension of the resulting packet-level features, M	64
The layer number of GNN	4
The multi-head number of attention	4
The dimension of the resulting flow-level features, S	256
Burst-threshold	1 s

over the 5 folds. The final validation loss is treated as the score of this model and is further used as the criterion for comparing with different hyperparameter settings. Finally, we choose the best model with the minimum average validation loss.

We use Adam optimizer to minimize the loss, with its initial learning rate of 0.001, mini-batch size of 64, and decay rate of 0.01. We adopted a 7:2:1 split ratio for training, validation, and test sets, respectively. We implemented our model with Pytorch and run each experiment on a single NVIDIA RTX 3090 GPU.

4.1.4. Evaluation metrics

In the classification task, commonly used evaluation indicators are Accuracy (AC), Precision (PR), Recall (RC), and F1-score. We calculate the number of true positive (TP), false positive (FP), true negative (TN), and false negative (FN) on the test set. The metrics are calculated as follows:

$$AC = \frac{TP + TN}{TP + FP + TN + FN} \quad (13)$$

$$PR = \frac{TP}{TP + FP} \quad (14)$$

$$RC = \frac{TP}{TP + FN} \quad (15)$$

$$F1 = 2 \cdot \frac{PR \times RC}{PR + RC} \quad (16)$$

4.2. Model sensitivity analysis

The results of sensitivity analysis are shown in Figs. 7 and 8. We choose F1-score, which takes into account both the precision and recall of the classification model, as our observation metrics.

(1) The Impact of Packet Number.

The number of packets directly impacts the size of the constructed Traffic Interaction Graph (TIG), i.e. the number of nodes in the graph. The results are illustrated in Fig. 7. It can be observed that as the number of packets increases, the model's performance initially shows an upward trend, followed by stabilization. Specifically, when the number of packets exceeds 15, the model's performance nearly reaches an optimal point. We classify traffic by learning the structure of the graph. Therefore, when the number of nodes in the graph reaches a certain threshold, it contains sufficient information to support effective learning.

(2) The Impact of Byte Number.

The number of bytes extracted from each packet primarily influences packet-level learning. While the number of bytes in the packet header remains small and consistent, our attention is focused on assessing the impact of varying the number of bytes in the payload on the model's performance. It can be observed from Fig. 8 that when using the first 400 bytes of a payload for modeling, the model's performance is better than with other settings. As the number of bytes continues to increase, more noise is introduced, which can adversely affect the model's learning performance.

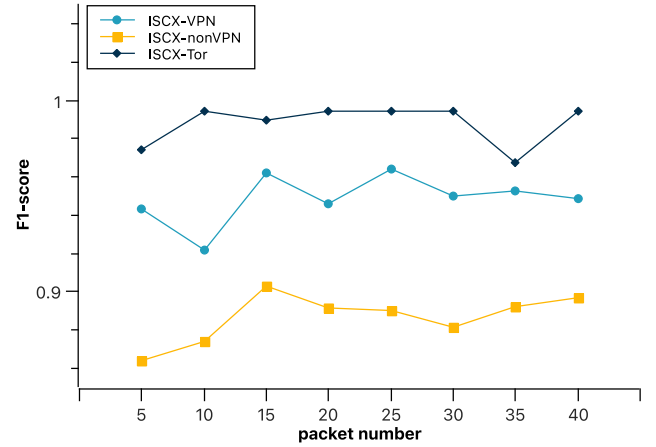


Fig. 7. The sensitivity analysis of packet number.

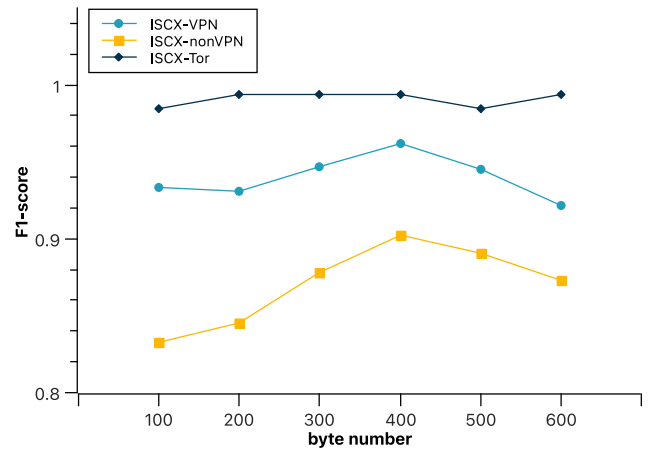


Fig. 8. The sensitivity analysis of byte number.

4.3. Comparison experiments

We use five traditional deep learning models and two GNN models as baselines:

(1) 1D-CNN: The raw packet bytes are transformed into grayscale graphs, and a CNN model is used for learning. We followed the implementation from [46].

(2) LSTM: The packets within the network flow exhibit temporal characteristics. We implemented an LSTM model with reference to [41] for traffic classification.

(3) FS-Net: FS-Net [28] uses a packet length sequence as input. It employs bi-directional GRU for feature encoding and introduces a reconstruction mechanism within an AutoEncoder to ensure the validity of the learned features.

(4) MIMETIC: MIMETIC [33] is a multimodal learning method which uses CNN and GRU respectively to learn the first 576 bytes of payload and informative protocol fields.

(5) App-Net: App-Net [34] is also a multimodal method which uses LSTM and CNN to learn sequence of packet lengths and payload bytes of the initial packet respectively.

(6) GraphDAPP: GraphDAPP [35] constructs the network flow as a Traffic Interaction Graph, using packet direction and packet length as node features to achieve traffic classification.

(7) FB-GNN: FB-GNN [36] proposed a graph construction method, which directly create edges based on the chronological relationship of packets among a flow and leverage the geometric learning model to learn the node, edge, and global features.

Table 4

Experimental results on ISCX datasets. The table presents the results of different models on three datasets.

Dataset	ISCX-VPN				ISCX-nonVPN				ISCX-Tor			
Model	AC	PR	RC	F1	AC	PR	RC	F1	AC	PR	RC	F1
1D-CNN	0.8359	0.8604	0.8242	0.8308	0.7871	0.7927	0.7960	0.7919	0.5814	0.4608	0.4366	0.4349
LSTM	0.9336	0.9321	0.9172	0.9240	0.7539	0.7928	0.7070	0.7306	0.7442	0.7120	0.7036	0.7048
FS-Net	0.9121	0.9117	0.9268	0.9179	0.7539	0.8021	0.7183	0.7413	0.8140	0.8574	0.8176	0.8187
MIMETIC	0.8047	0.8559	0.8352	0.8387	0.6406	0.7115	0.6015	0.6086	0.2857	0.0688	0.1578	0.0925
App-Net	0.9609	0.9464	0.9573	0.9495	0.8711	0.8779	0.8760	0.8760	0.7143	0.5192	0.5374	0.5168
GraphDAPP	0.6585	0.5028	0.4148	0.4276	0.3809	0.3454	0.3363	0.2634	0.5272	0.3359	0.2370	0.2292
FB-GNN	0.8431	0.8293	0.8141	0.8184	0.6503	0.6558	0.6541	0.6391	0.4286	0.2381	0.3136	0.2581
DE-GNN	0.9688	0.9659	0.9594	0.9624	0.8984	0.9110	0.9007	0.9048	0.9872	0.9943	0.9939	0.9905

The comparison results on the ISCX datasets are presented in Table 4. Based on the results, we can draw the following conclusions: (1) DE-GNN achieves the best performance compared to several baselines across all four metrics. This comprehensive performance demonstrates the effectiveness of our method. (2) Neither the 1D-CNN model nor the LSTM model delivers satisfactory results. As previously mentioned, encrypted traffic conceals the original content, resulting in limited extraction of effective information by both CNN and RNN models. Furthermore, the LSTM outperforms the 1D-CNN, which can be attributed to the LSTM's capacity to capture temporal information within the network flow, which plays a crucial role in the context of encrypted traffic. (3) FS-Net employs the packet length sequence for end-to-end classification and achieves performance similar to the LSTM. This outcome can be attributed to the fact that both FS-Net and LSTM focus on learning temporal information. (4) The multimodal methods MIMETIC and App-Net utilize multiple modalities in their learning processes. Their performance shows significant variability across different datasets, particularly on the Tor dataset. The limited number of samples in the Tor dataset contributes to overfitting issues, impacting performance outcomes. (5) Compared with the two GNN-based methods, our method outperforms both in all aspects. GraphDAPP also uses the Traffic Interaction Graph (TIG) for classification but does not yield satisfactory results. The main reason for this lies in its use of packet length and packet direction as node features. However, the presence of technologies such as packet fragmentation, VPN, and Tor introduces significant interference with packet lengths, consequently leading to performance degradation. FB-GNN is unstable on different datasets. The main reason is that their graphs are constructed according to time series, which means that graphs with the same number of nodes have the same structure, significantly reducing performance stability.

4.4. Ablation experiments

To investigate the effect of each component of the DE-GNN model, we also perform the ablation test of DE-GNN in terms of:

(1) Dual-embedding: Instead of using both the header and payload, we use only one of them. This method is denoted as **All-h** (only header) and **All-p** (only payload), respectively.

(2) Packet-level learning: We discard the packet-level learning and directly use the original byte value of the packet as the features of the nodes in the graph. This method is denoted as **All-packet** (packet).

(3) Graph neural networks: To illustrate the effectiveness of the Graph, instead of using TIG and GNN, we learn directly with CNN or LSTM in the flow-level learning phase, called **All-cnn** and **All-lstm**, respectively.

(4) Adaptive deep feature fusion: Instead of using ADFF for the feature fusion of header and payload, we directly concatenate them, denoted as **All-adff** (ADFF).

The results of the ablation tests are presented in Tables 5 and 6. Several key observations can be made. First, it is evident that packet headers are more influential in classification compared to packet

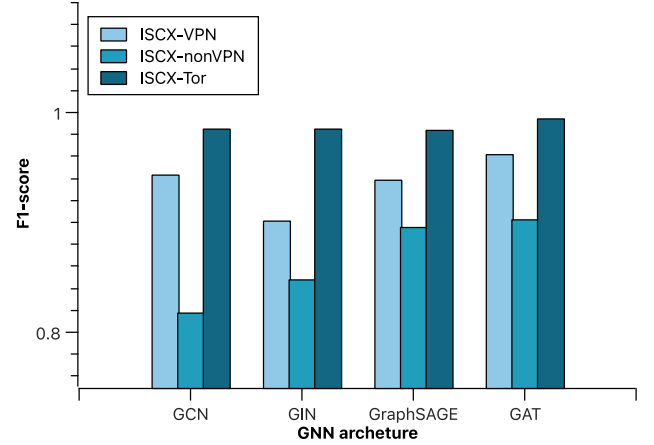


Fig. 9. The result of different GNN architectures. The F1-score results for employing different GNN architectures in the flow-level learning stage are presented in the figure.

payloads. Moreover, different datasets exhibit varying degrees of the importance of headers and payloads. For example, when switching from using headers to payloads, the F1-score decreases by 12.53% on the ISCX-VPN dataset and by 0.19% on the ISCX-Tor dataset. Second, the omission of packet-level learning has the most significant negative impact on model performance. This is particularly noticeable in the ISCX-Tor dataset, where the F1 score dropped by 47.38%. When using the raw byte values directly as node features, the data contains considerable noise and redundant information, making it challenging for the model to learn effectively. By subjecting the original byte data to deep feature extraction through PacketCNN, packet-level information can be more efficiently represented, facilitating further learning. Third, TIG and GNN outperform CNN and LSTM in terms of performance improvement. Constructing the traffic interaction graph allows for the acquisition of not only information from the original bytes but also information related to traffic interactions. Additionally, the application of adaptive weight fusion proves to enhance model performance.

Analyzing the complexity of ablation models reveals that All-h, All-p, All-packet, and All-adff effectively reduce both parameters and FLOPS, as they only encompass a subset of the original model. Notably, All-h achieves a notable reduction in model complexity while ensuring limited performance degradation. Conversely, the other models increase complexity due to changing the structures, without yielding any performance improvements.

4.5. GNN architecture variants study

To demonstrate the effectiveness of GAT for learning TIG, we select some classical GNN architectures as variants (i.e., GCN [47], GIN [48], GraphSAGE [49]) for comparison.

Table 5
Ablation test results on ISCX datasets.

Dataset	ISCX-VPN				ISCX-nonVPN				ISCX-Tor			
Model	AC	PR	RC	F1	AC	PR	RC	F1	AC	PR	RC	F1
All-h	0.9492	0.9415	0.9472	0.9418	0.8750	0.8779	0.8828	0.8794	0.9744	0.9563	0.9815	0.9666
All-p	0.8438	0.8329	0.8544	0.8165	0.7402	0.7572	0.7158	0.7258	0.9487	0.9593	0.9762	0.9647
All-packet	0.8672	0.8719	0.8497	0.8595	0.7344	0.7911	0.6981	0.7163	0.6154	0.5501	0.5942	0.5167
All-cnn	0.9219	0.8999	0.9171	0.9017	0.8887	0.8998	0.8875	0.8918	0.9487	0.9583	0.9696	0.9588
All-ilstm	0.9492	0.9338	0.9568	0.9443	0.8633	0.8660	0.8631	0.8645	0.9231	0.9045	0.9449	0.9125
All-adff	0.9531	0.9475	0.9508	0.9482	0.8730	0.8736	0.8963	0.8831	0.9744	0.9631	0.9815	0.9696
All	0.9688	0.9659	0.9594	0.9624	0.8984	0.9110	0.9007	0.9048	0.9872	0.9943	0.9939	0.9905

Table 6
Model FLOPs and parameters of ablation test.

Model	FLOPs(M)	Parameters(M)
All-h	3.3e+1	1.3e-1
All-p	4.4e+2	1.3e-1
All-packet	2.4e+0	5.1e-1
All-cnn	4.8e+2	1.4e+0
All-ilstm	4.9e+2	1.9e+0
All-adff	4.7e+2	2.6e-1
All	4.7e+2	6.2e-1

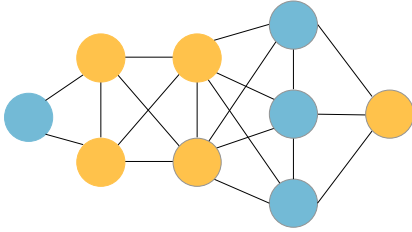


Fig. 10. The fully connected variant of TIG. We connect all nodes between adjacent bursts to each other.

As shown in Fig. 9, our findings suggest that the Graph Attention Network (GAT) outperformed other variants. This superiority is due to the attention mechanism integrated into GAT. As previously explained, GAT can evaluate the importance of different neighboring nodes near a given node and utilize this information in node feature propagation. This feature effectively reduces the influence of less important nodes during the information transfer process within the graph.

4.6. Structure of the Traffic Interaction Graph (TIG)

In the construction of the TIG, the process begins by segmenting bursts and then establishing connections between nodes using intra-burst and inter-burst mechanisms, as outlined in Algorithm 1. This approach facilitates the exchange of node information within and between bursts during the graph neural network learning process. To gain a deeper understanding of the rationale behind TIG's structure, we create a fully connected variant of TIG. In this version, the nodes of two adjacent bursts are connected to each other, as opposed to connecting the start and end nodes. This is illustrated in Fig. 10.

The results are depicted in Fig. 11. It is evident that employing a fully connected graph does not yield improvement; in fact, it led to a decrease in performance. This outcome can be attributed to several reasons:

(1) Increased Computational Load: Connecting every node in the graph results in a substantial increase in computational demands. This additional load affects the efficiency of the learning process. (2) Redundancy of Information Transmission: In the learning process of a multi-layer graph neural network, full connectivity can introduce redundancy in the transmission of information. This redundancy can

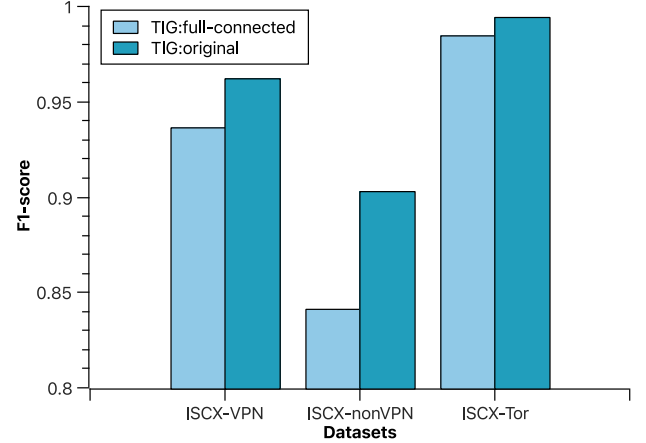


Fig. 11. The result of different graph structure.

hinder the network's ability to extract meaningful features. (3) Complexity of Learning: The goal of using a graph neural network is to understand the structure and relationships within the entire graph and ultimately derive a comprehensive feature representation of the graph. When the graph becomes excessively complex due to full connectivity, the learning process also becomes intricate, making it more challenging to capture essential patterns.

4.7. Model complexity analysis

To comprehensively evaluate the trade-off between model performance and model complexity, we present the floating point operations (FLOPs) and the model size of all baselines and our model. The result is shown in Table 7.

From Tables 7 and 4, we can draw a conclusion that DE-GNN achieves the most significant improvement on public datasets with relatively slight model complexity increasing. It can be observed from Table 7 that DE-GNN has the second-fewest number of parameters among different methods. The significant advantage of having fewer parameters is the lower cost for transferring the model for further studies or deployment. However, conversely, DE-GNN requires more floating-point computations. This is mainly due to the sophisticated structure: (1) We use raw bytes as the input to the model and design a multi-step information extraction process based on the hierarchical structure, which increases the computation. By reducing the number of input packets and bytes, we can effectively reduce FLOPs and obtain an acceptable drop in accuracy. (2) We treat the packet header and the payload separately, using two identical neural network structures to learn them, which also leads to an increase in computation. However, it should be noted that for the processing of packet headers and payloads, we can handle them in parallel.

Table 7
Model FLOPs and parameters.

Model	FLOPs(M)	Parameters(M)
1D-CNN	1.9e+1	5.8e+0
LSTM	3.4e+1	2.4e+0
FS-NET	1.0e+2	3.2e+0
MIMETIC	3.4e+1	2.3e+0
App-Net	5.6e+2	2.1e+0
GraphDAPP	2.4e+2	2.5e+2
FB-GNN	5.0e+2	9.5e+0
DE-GNN	4.7e+2	6.2e+1

5. Limitations and future works

In this section, we further discuss the limitations as well as the future work of our approach.

A restricted graph structure. In this paper, we employ the concept of “burst” to segment network flows and build a Traffic Interaction Graph (TIG) for network flow representation. While the TIG effectively represents the interactive information within the network flow, it does have some limitations in terms of information transmission within the graph. Specifically, transferring information between non-adjacent bursts requires multiple layers of graph neural networks and cannot be achieved in a single learning step. We acknowledge these limitations and intend to refine the graph structure in our future work.

Multiple flows detection. Although the main emphasis of this paper is on classifying individual network flows, it is essential to acknowledge that real-world activities frequently involve multiple network flows. In our future research, we plan to investigate methods for leveraging the relationships between multiple network flows to improve traffic classification in these intricate scenarios.

6. Conclusion

In this paper, we introduce DE-GNN, a deep learning-based model designed for the fine-grained classification of encrypted network traffic. Capitalizing on the inherent hierarchical structure of network traffic, specifically, the byte-packet-flow hierarchy, DE-GNN delves deep into encrypted traffic to extract meaningful information. Our approach commences by encoding the raw bytes and utilizes PacketCNN to acquire packet-level representations of the traffic. Subsequently, we represent network flows as Traffic Interaction Graph (TIG) and employ Graph Neural Networks (GNN) for flow-level learning. Given the distinction between packet headers and payloads, we apply the same process separately to both packet headers and payloads as described above. Finally, the adaptive weighted feature fusion (ADFF) mechanism is employed to combine the features extracted from packet headers and payloads. Experimental results on two public datasets demonstrate the effectiveness of DE-GNN.

CRedit authorship contribution statement

Xinbo Han: Writing – original draft, Software, Methodology, Investigation, Conceptualization. **Guizhong Xu:** Validation, Software, Investigation. **Meng Zhang:** Resources, Methodology. **Zheng Yang:** Writing – review & editing, Formal analysis. **Ziyang Yu:** Resources. **Weiqing Huang:** Validation. **Chen Meng:** Writing – review & editing, Methodology.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

This research has been partially funded by the Institute of Information Engineering, Chinese Academy of Science, Project 2021YFB29101-10, and is supported by the University of Chinese Academy of Science.

References

- [1] S. Rezaei, X. Liu, Deep learning for encrypted traffic classification: An overview, *IEEE Commun. Mag.* 57 (5) (2019) 76–81, <http://dx.doi.org/10.1109/mcom.2019.1800819>.
- [2] R. Sharma, S. Dangi, P. Mishra, A comprehensive review on encryption based open source cyber security tools, in: 2021 6th International Conference on Signal Processing, Computing and Control, ISPPC, IEEE, 2021, pp. 614–619, <http://dx.doi.org/10.1109/isppc53510.2021.9609369>.
- [3] E. Ramadhani, Anonymity communication VPN and Tor: a comparative study, in: *Journal of Physics: Conference Series*, vol. 983, IOP Publishing, 2018, 012060, <http://dx.doi.org/10.1088/1742-6596/983/1/012060>.
- [4] T. Nelms, R. Perdisci, M. Ahamad, {ExeScent}: Mining for new {C&C} domains in live networks with adaptive control protocol templates, in: 22nd USENIX Security Symposium, USENIX Security 13, 2013, pp. 589–604, <http://dx.doi.org/10.5555/2534766.2534817>.
- [5] J. Su, S. Chen, B. Han, C. Xu, X. Wang, A 60Gbps DPI prototype based on memory-centric FPGA, in: *Proceedings of the 2016 ACM SIGCOMM Conference*, 2016, pp. 627–628, <http://dx.doi.org/10.1145/2934872.2959079>.
- [6] W. Melo, P. Lopes, R. Antonello, S. Fernandes, D. Sadok, On the performance of DPI signature matching with dynamic priority, in: 2014 IEEE Symposium on Computers and Communications, ISCC, IEEE, 2014, pp. 1–6, <http://dx.doi.org/10.1109/ISCC.2014.6912553>.
- [7] E. Papadogiannaki, S. Ioannidis, A survey on encrypted network traffic analysis applications, techniques, and countermeasures, *ACM Comput. Surv.* 54 (6) (2021) 1–35, <http://dx.doi.org/10.1145/3457904>.
- [8] J. Jarmoc, SSL/TLS interception proxies and transitive trust, 2012, URL <https://api.semanticscholar.org/CorpusID:9485779>.
- [9] J. Han, S. Kim, J. Ha, D. Han, Sgx-box: Enabling visibility on encrypted traffic using a secure middlebox module, in: *Proceedings of the First Asia-Pacific Workshop on Networking*, 2017, pp. 99–105, <http://dx.doi.org/10.1145/3106989.3106994>.
- [10] B. Trach, A. Krohmer, F. Gregor, S. Arnautov, P. Bhatotia, C. Fetzer, Shieldbox: Secure middleboxes using shielded execution, in: *Proceedings of the Symposium on SDN Research*, 2018, pp. 1–14, <http://dx.doi.org/10.1145/3185467.3185469>.
- [11] D. Goltzsche, S. Rüsch, M. Nieke, S. Vaucher, N. Weichbrodt, V. Schiavoni, P.-L. Aublin, P. Cosa, C. Fetzer, P. Felber, et al., Endbox: Scalable middlebox functions using client-side trusted execution, in: 2018 48th Annual IEEE/IFIP International Conference on Dependable Systems and Networks, DSN, IEEE, 2018, pp. 386–397, <http://dx.doi.org/10.1109/dsn.2018.00048>.
- [12] O. Salman, I.H. Elhaji, A. Kayssi, A. Chehab, A review on machine learning-based approaches for Internet traffic classification, *Ann. Telecommun.* 75 (2020) 673–710, <http://dx.doi.org/10.1007/s12243-020-00770-7>.
- [13] S. Dong, Multi class SVM algorithm with active learning for network traffic classification, *Expert Syst. Appl.* 176 (2021) 114885, <http://dx.doi.org/10.1016/j.eswa.2021.114885>.
- [14] G. Aceto, D. Ciuonzo, A. Montieri, A. Pescapé, Toward effective mobile encrypted traffic classification through deep learning, *Neurocomputing* 409 (2020) 306–315, <http://dx.doi.org/10.1016/j.neucom.2020.05.036>.
- [15] P. Wang, X. Chen, F. Ye, Z. Sun, A survey of techniques for mobile service encrypted traffic classification using deep learning, *IEEE Access* 7 (2019) 54024–54033, <http://dx.doi.org/10.1109/access.2019.2912896>.
- [16] G. Aceto, D. Ciuonzo, A. Montieri, A. Pescapé, DISTILLER: Encrypted traffic classification via multimodal multitask deep learning, *J. Netw. Comput. Appl.* 183 (2021) 102985, <http://dx.doi.org/10.1016/j.jnca.2021.102985>.
- [17] J. Lan, X. Liu, B. Li, Y. Li, T. Geng, DarknetSec: A novel self-attentive deep learning method for darknet traffic classification and application identification, *Comput. Secur.* 116 (2022) 102663, <http://dx.doi.org/10.1016/j.cose.2022.102663>.
- [18] H. Zhang, L. Yu, X. Xiao, Q. Li, F. Mercaldo, X. Luo, Q. Liu, TFE-GNN: A temporal fusion encoder using graph neural networks for fine-grained encrypted traffic classification, in: *Proceedings of the ACM Web Conference 2023*, 2023, pp. 2066–2075, <http://dx.doi.org/10.1145/3543507.3583227>.
- [19] X. Yun, J. Xie, S. Li, Y. Zhang, P. Sun, Detecting unknown HTTP-based malicious communication behavior via generated adversarial flows and hierarchical traffic features, *Comput. Secur.* 121 (2022) 102834, <http://dx.doi.org/10.1016/j.cose.2022.102834>.

- [20] M. Shen, Y. Liu, L. Zhu, X. Du, J. Hu, Fine-grained webpage fingerprinting using only packet length information of encrypted traffic, *IEEE Trans. Inf. Forensics Secur.* 16 (2020) 2046–2059, <http://dx.doi.org/10.1109/tifs.2020.3046876>.
- [21] J. Xing, C. Wu, Detecting anomalies in encrypted traffic via deep dictionary learning, in: *IEEE INFOCOM 2020-IEEE Conference on Computer Communications Workshops, INFOCOM WKSHPs*, IEEE, 2020, pp. 734–739, <http://dx.doi.org/10.1109/INFOCOMWKSHPs50562.2020.9162940>.
- [22] V.F. Taylor, R. Spolaor, M. Conti, I. Martinovic, Appscanner: Automatic fingerprinting of smartphone apps from encrypted network traffic, in: *2016 IEEE European Symposium on Security and Privacy, EuroS&P*, IEEE, 2016, pp. 439–454, <http://dx.doi.org/10.1109/eurosp.2016.40>.
- [23] C. Liu, Z. Cao, G. Xiong, G. Gou, S.-M. Yiu, L. He, Mampf: Encrypted traffic classification based on multi-attribute markov probability fingerprints, in: *2018 IEEE/ACM 26th International Symposium on Quality of Service (IWQoS)*, IEEE, 2018, pp. 1–10, <http://dx.doi.org/10.1109/iwqos.2018.8624124>.
- [24] T. Van Ede, R. Bortolameotti, A. Continella, J. Ren, D.J. Dubois, M. Lindorfer, D. Choffnes, M. van Steen, A. Peter, Flowprint: Semi-supervised mobile-app fingerprinting on encrypted network traffic, in: *Network and Distributed System Security Symposium, NDSS*, Vol. 27, 2020, <http://dx.doi.org/10.14722/ndss.2020.24412>.
- [25] V.F. Taylor, R. Spolaor, M. Conti, I. Martinovic, Robust smartphone app identification via encrypted network traffic analysis, *IEEE Trans. Inf. Forensics Secur.* 13 (1) (2017) 63–78, <http://dx.doi.org/10.1109/tifs.2017.2737970>.
- [26] A. Saber, B. Fergani, M. Abbas, Encrypted traffic classification: Combining over- and under-sampling through a pca-svm, in: *2018 3rd International Conference on Pattern Analysis and Intelligent Systems, PAIS*, IEEE, 2018, pp. 1–5, <http://dx.doi.org/10.1109/pais.2018.8598480>.
- [27] B. Anderson, S. Paul, D. McGrew, Deciphering malware's use of TLS (without decryption), *J. Comput. Virol. Hacking Tech.* 14 (2018) 195–211, <http://dx.doi.org/10.1007/s11416-017-0306-6>.
- [28] C. Liu, L. He, G. Xiong, Z. Cao, Z. Li, Fs-net: A flow sequence network for encrypted traffic classification, in: *IEEE INFOCOM 2019-IEEE Conference on Computer Communications*, IEEE, 2019, pp. 1171–1179, <http://dx.doi.org/10.1109/infocom.2019.8737507>.
- [29] T. Shapira, Y. Shavitt, FlowPic: A generic representation for encrypted traffic classification and applications identification, *IEEE Trans. Netw. Serv. Manag.* 18 (2) (2021) 1218–1232, <http://dx.doi.org/10.1109/tnsn.2021.3071441>.
- [30] X. Liu, J. You, Y. Wu, T. Li, L. Li, Z. Zhang, J. Ge, Attention-based bidirectional GRU networks for efficient HTTPS traffic classification, *Inform. Sci.* 541 (2020) 297–315, <http://dx.doi.org/10.1016/j.ins.2020.05.035>.
- [31] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, J. Yu, Et-ber: A contextualized datagram representation with pre-training transformers for encrypted traffic classification, in: *Proceedings of the ACM Web Conference 2022*, 2022, pp. 633–642, <http://dx.doi.org/10.1145/3485447.3512217>.
- [32] M. Lotfollahi, M. Jafari Siavoshani, R. Shirali Hossein Zade, M. Saberian, Deep packet: A novel approach for encrypted traffic classification using deep learning, *Soft Comput.* 24 (3) (2020) 1999–2012, <http://dx.doi.org/10.1007/s00500-019-04030-2>.
- [33] G. Aceto, D. Ciunzio, A. Montieri, A. Pescapè, MIMETIC: Mobile encrypted traffic classification using multimodal deep learning, *Comput. Netw.* 165 (2019) 106944, <http://dx.doi.org/10.1016/j.comnet.2019.106944>.
- [34] X. Wang, S. Chen, J. Su, App-net: A hybrid neural network for encrypted mobile traffic classification, in: *IEEE INFOCOM 2020-IEEE Conference on Computer Communications Workshops, INFOCOM WKSHPs*, IEEE, 2020, pp. 424–429, <http://dx.doi.org/10.1109/INFOCOMWKSHPs50562.2020.9162891>.
- [35] M. Shen, J. Zhang, L. Zhu, K. Xu, X. Du, Accurate decentralized application identification via encrypted traffic analysis using graph neural networks, *IEEE Trans. Inf. Forensics Secur.* 16 (2021) 2367–2380, <http://dx.doi.org/10.1109/tifs.2021.3050608>.
- [36] T.-L. Huoh, Y. Luo, P. Li, T. Zhang, Flow-based encrypted network traffic classification with graph neural networks, *IEEE Trans. Netw. Serv. Manag.* (2022) <http://dx.doi.org/10.1109/TNSM.2022.3227500>.
- [37] P.W. Battaglia, J.B. Hamrick, V. Bapst, A. Sanchez-Gonzalez, V. Zambaldi, M. Malinowski, A. Tacchetti, D. Raposo, A. Santoro, R. Faulkner, et al., Relational inductive biases, deep learning, and graph networks, 2018, <http://dx.doi.org/10.48550/arXiv.1806.01261>, arXiv preprint [arXiv:1806.01261](https://arxiv.org/abs/1806.01261).
- [38] Y. Kim, Convolutional neural networks for sentence classification, 2014, <http://dx.doi.org/10.48550/arXiv.1408.5882>, arXiv preprint [arXiv:1408.5882](https://arxiv.org/abs/1408.5882).
- [39] K. Al-Naami, S. Chandra, A. Mustafa, L. Khan, Z. Lin, K. Hamlen, B. Thuraisingham, Adaptive encrypted traffic fingerprinting with bi-directional dependence, in: *Proceedings of the 32nd Annual Conference on Computer Security Applications*, 2016, pp. 177–188, <http://dx.doi.org/10.1145/2991079.2991123>.
- [40] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Lio, Y. Bengio, et al., Graph attention networks, *stat* 1050 (20) (2017) 10–48550, <http://dx.doi.org/10.48550/arXiv.1710.10903>.
- [41] K. Lin, X. Xu, F. Xiao, MFFusion: A multi-level features fusion model for malicious traffic detection based on deep learning, *Comput. Netw.* 202 (2022) 108658, <http://dx.doi.org/10.1016/j.comnet.2021.108658>.
- [42] Q. Zhang, H. Li, Z. Sun, T. Tan, Deep feature fusion for iris and periocular biometrics on mobile devices, *IEEE Trans. Inf. Forensics Secur.* 13 (11) (2018) 2897–2912, <http://dx.doi.org/10.1109/tifs.2018.2833033>.
- [43] Y. Li, L. Liu, H. Qin, S. Deng, M.A. El-Yacoubi, G. Zhou, Adaptive deep feature fusion for continuous authentication with data augmentation, *IEEE Trans. Mob. Comput.* (2022) <http://dx.doi.org/10.1109/tmc.2022.3186614>.
- [44] G. Draper-Gil, A.H. Lashkari, M.S.I. Mamun, A.A. Ghorbani, Characterization of encrypted and vpn traffic using time-related, in: *Proceedings of the 2nd International Conference on Information Systems Security and Privacy, ICISSP*, 2016, pp. 407–414, <http://dx.doi.org/10.5220/0005740704070414>.
- [45] A.H. Lashkari, G.D. Gil, M.S.I. Mamun, A.A. Ghorbani, Characterization of tor traffic using time based features, in: *International Conference on Information Systems Security and Privacy*, Vol. 2, SciTePress, 2017, pp. 253–262, <http://dx.doi.org/10.5220/0006105602530262>.
- [46] W. Wang, M. Zhu, J. Wang, X. Zeng, Z. Yang, End-to-end encrypted traffic classification with one-dimensional convolution neural networks, in: *2017 IEEE International Conference on Intelligence and Security Informatics, ISI*, IEEE, 2017, pp. 43–48, <http://dx.doi.org/10.1109/ISI.2017.8004872>.
- [47] T.N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, 2016, <http://dx.doi.org/10.48550/arXiv.1609.02907>, arXiv preprint [arXiv:1609.02907](https://arxiv.org/abs/1609.02907).
- [48] K. Xu, W. Hu, J. Leskovec, S. Jegelka, How powerful are graph neural networks? 2018, <http://dx.doi.org/10.48550/arXiv.1810.00826>, arXiv preprint [arXiv:1810.00826](https://arxiv.org/abs/1810.00826).
- [49] W.L. Hamilton, R. Ying, J. Leskovec, Inductive representation learning on large graphs, 2018, <http://dx.doi.org/10.48550/arXiv.1706.02216>, arXiv:1706.02216.

Xinbo Han received bachelor's degree from Taiyuan University Of Technology, in 2020. He is presently pursuing the Ph.D. degree at School of Cyber Security, University of Chinese Academy of Sciences. His current research interests include network attack detection, encrypted traffic classification and deep learning.

Guizhong Xu received master's degree from Communication University Of China. He is now an senior engineer in the Information Science Academy of China Electronics Technology Group Corporation. His research interests include anomaly detection and network traffic analysis.

Meng Zhang received Ph.D. degree from University of Chinese Academy of Sciences. He is a research professor, Ph.D. supervisor of Institute of Information Engineering, Chinese Academy of Sciences. His research interests include signal processing theory and technology, electromagnetic acoustic and optical detection and protection.

Zheng Yang received master's degree from Beijing University of Posts and Telecommunications, Beijing, China. He is now an associate senior engineer in the institute of information engineering, Chinese Academy of Sciences. His research interests include moving target defense.

Ziyang Yu received bachelor's degree from Huazhong University Of Science And Technology, in 2018. He is presently pursuing the Ph.D. degree at School of Cyber Security, University of Chinese Academy of Sciences. His current research interests include APT attack detection and investigation based on provenance graph.



Weiqing Huang is a research professor, Ph.D. supervisor of Institute of Information Engineering, Chinese Academy of Sciences. He is also a professor in the School of Cyber Security, University of Chinese Academy of Sciences. His research interests include signal processing theory and technology, electromagnetic acousto-optical detection and protection.



Chen Meng received master's degree from Beijing University of Posts and Telecommunications, Beijing, China. He is now an engineer in the institute of information engineering, Chinese Academy of Sciences. His research interests include encrypted traffic classification.